applied sciences

MDPI

Article

Optimization Strategies Applied to Deep Learning Models for Image Steganalysis: Application of Pruning, Quantization and Weight Clustering

Gabriel Ferreira, Manoel Henrique da Nóbrega Marinho, Verusca Severo * and Francisco Madeiro

Polytechnic School of Pernambuco, University of Pernambuco, Recife 50720-001, Brazil; gaf@poli.br (G.F.); marinho75@poli.br (M.H.d.N.M.); madeiro@poli.br (F.M.)

* Correspondence: verusca.severo@poli.br

Abstract: Image steganalysis methods aim at detecting whether there exist hidden messages in images. Deep learning (DL) models have been proposed to enhance steganography detection. These models occupy a large amount of memory and, for this reason, should be optimized when the scenario involves resource-limited devices and systems. This work addresses different deep learning model optimization strategies, namely model pruning, quantization and weight clustering, applied to a deep learning model that presents competitive accuracy results in image steganalysis and belongs to the family of DL models with smaller memory requirements. The results show that the use of optimization schemes can lead to similar or even better accuracy compared to the original model (without the use of optimization schemes), while requiring less memory to store the model. Different scenarios are simulated for each optimization technique, and, finally, quantization is combined with pruning. For dynamic range quantization (DRQ), we achieve models that can save approximately 72% of storage. For FP16 quantization, we obtain better accuracy results and a model with approximately 50% less memory consumption. By applying weight clustering, we also achieve compressed models that can save more than 72% of storage space and lead to better accuracy for some scenarios. Using the combination of pruning and quantization, smaller models in terms of memory requirements are obtained.

Keywords: steganalysis; deep neural networks; pruning; quantization; weight clustering

Academic Editor: Pedro Couto

Received: 4 April 2025
Accepted: 17 April 2025
Published: 22 April 2025

Citation: Ferreira, G.; Marinho, M.H.d.N, Severo, V.; Madeiro, F. Optimization Strategies Applied to Deep Learning Models for Image Steganalysis: Application of Pruning, Quantization and Weight Clustering. *Appl. Sci.* **2025**, *15*, 4632. https://doi.org/10.3390/app15094632

1. Introduction

Over the years, people have obtained greater access to communication devices and networks, resulting in more information sent over the internet, such as text messages, audio, images, and videos. Since these types of information can be changed and eavesdropped upon, encryption and data hiding techniques, such as steganography and watermarking, can be applied to achieve both malicious and good objectives [1].

Steganography is a subarea of data hiding with the objective of sending an occult message through a communication channel [2]. Steganalysis is a counter mechanism that focuses on discovering whether there is the presence of hidden information, thus counteracting steganography [3].

In the digital era, steganography methods have been used for malicious objectives, e.g., to leak information [4,5], for terrorism purposes [6,7], for cyber warfare [8], and for malware propagation [9–11]. As a countermeasure, steganalysis schemes have been employed to track criminal activities on the internet [12,13], to combat cyber warfare [14], in the detection of malware [15,16], and in forensics [13,17,18].

Steganography methods can be classified via the domain used to embed secret information. While spatial domain techniques consist of hiding information by changing the image pixel values, transform domain methods are based on embedding messages in the transform coefficients of images [19]. Among the steganography algorithms adopted to evaluate the performance of steganalysis in the spatial domain are Highly Undetectable steGO (HUGO) [20], wavelet-obtained weights (WOW) [21], spatial universal wavelet relative (S-UNIWARD) [22], and high-pass, low-pass, and low-pass (HILL) [23].

Steganalysis methods can be classified into specific steganalysis, which means that the technique is responsible for discovering whether there is information concealed in a specific steganography method, or universal/black-box/blind steganalysis, which can detect the existence of information embedded by any steganographic scheme, mostly through computational intelligence techniques [24]. These methods are used to discover if there is hidden communication in the most popular types of media [25], such as audio [26], video [27], text [28], and images [29].

In recent years, deep learning models for image steganalysis have been developed to improve the detection of hidden messages in images [30–32]. They have been proposed as an alternative to traditional methods, seeking to achieve better performance in terms of accuracy in detecting the presence of steganography. Different scenarios have been evaluated by changing the payload capacity of the chosen steganographic algorithm and the arrangement of the test, validation, and train image sets [33]. Reinel et al. [34] proposed a deep learning network applied to image steganalysis called GBRAS-Net, which achieves competitive results in terms of accuracy and belongs to the group of models that require less memory when compared to previously proposed models.

Deep learning models can have a structure with millions of parameters and many activation functions to be stored, resulting in concerns regarding the implementation of these models in resource-limited devices. To overcome this issue, efficient and smaller networks have been proposed recently by applying optimization techniques such as model pruning, quantization, and weight clustering, without a significant reduction in terms of accuracy [35–37].

This work aims to assess the effects of deep learning model compression strategies, namely quantization, pruning, and weight clustering, in a deep learning model applied to image steganalysis (GBRAS-Net). An evaluation in terms of accuracy and memory requirements is carried out for a wide variety of compression strategies.

The main scientific contribution of this research lies in the implementation of optimization strategies in a deep learning model for image steganalysis, specifically through the application of pruning, quantization, and weight clustering. These techniques not only significantly reduce the memory requirements of the model, making it more suitable for devices with limited resources, but, in some scenarios, also maintain or even improve the accuracy compared to the original model. Furthermore, this innovative approach combines model compression methods with the critical field of information security, offering an optimized solution for efficient steganalysis systems. The relevance of this research extends to areas such as cybersecurity, digital forensics, and data protection, providing valuable and original insights into how to apply optimization techniques in deep neural networks for image steganalysis.

The remainder of this paper is organized as follows. In Section 2, we address the model optimization techniques. In Sections 3 and 4, we present the related works and methodology, respectively. In Section 5, we present the simulation results for the original and compressed models applied to S-UNIWARD and WOW. In Section 6, we present the conclusions and future work.

2. Model Optimization

2.1. Connection Pruning

Pruning is an optimization technique that aims to reduce the size of a model by removing connections, neurons, or parameters that are not relevant. Connection pruning consists of removing weights whose values are zero or close to zero to reduce the memory requirements of the entire network. The pruned model can be retrained and can even achieve superior accuracy compared to the original model [38]. Figure 1 shows a basic example of a model before pruning, i.e., with all neurons connected, and the resulting pruned model with fewer connections [39].

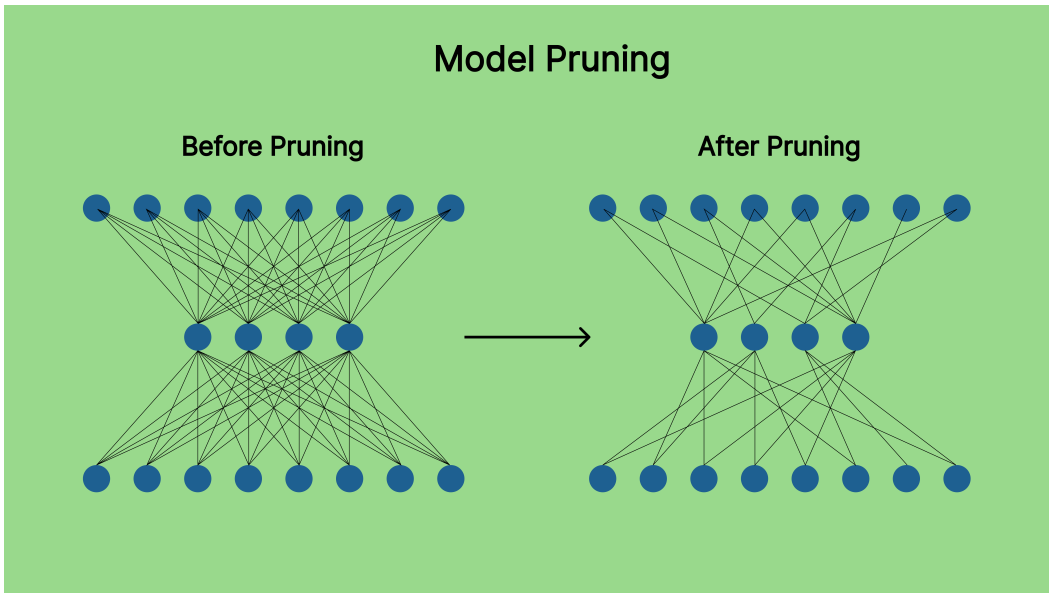

Figure 1. Example of pruning.

Magnitude-based pruning is one of the most common pruning techniques and has achieved great results in terms of reducing the memory requirements of machine learning models without compromising the accuracy of the scheme [38,40]. This technique consists of eliminating weights whose values are lower than a chosen threshold for a trained model. Pruning can also be implemented by defining the pruning ratio (PR) of the model, which refers to the percentage of weights to be zeroed in an entire network or the specific layers to be pruned by applying a pruning schedule [41].

2.2. Quantization

Quantization is a model optimization technique that has the objective of reducing the size of a DL model by converting weight values to a lower-precision bit representation, which is usually from 32 bits to 16 bits or 8 bits. Furthermore, it is possible to represent floating-point numbers in binary [42]. Quantization can be divided into two main categories: post-training quantization (PTQ) [43] and quantization-aware training (QAT) [44].

The PTQ technique is applied to a trained model with default values of representation—commonly 32-bit floating-point values. These values can be quantized, e.g., from 32-bit floating points (FP32) to 16-bit floating points (FP16) or 8-bit integers (INT8), without the need to retrain the model [45].

Figure 2 shows the FP32, FP16, FP8, and INT8 value representations on a computer system. It is observed that different representations use different numbers of bits. The precision is related to the number of bits used to represent a value. The more precise the representation is, the greater the amount of memory that has to be allocated to store the model.

The floating-point representation consists of three parts: the sign, exponent, and mantissa. The sign, represented in red, is the most significant bit and can either represent a positive or negative value (0 means positive and 1 means negative). The set of binary digits represented in green in the image is the exponent; it is responsible for defining the power to which the base 2 is raised. Occasionally, the exponent is called the signed or unbiased exponent. The other bits, represented in blue, are the mantissa and define the fractional part of the number [46]. For 8-bit integer representations, there is no exponent and the values can be represented from −128 to 127. Figure 3 shows dynamic range quantization (DRQ) from FP32 to INT8 representations. The values after conversion are less precise, but this results in less information to be stored in a system.

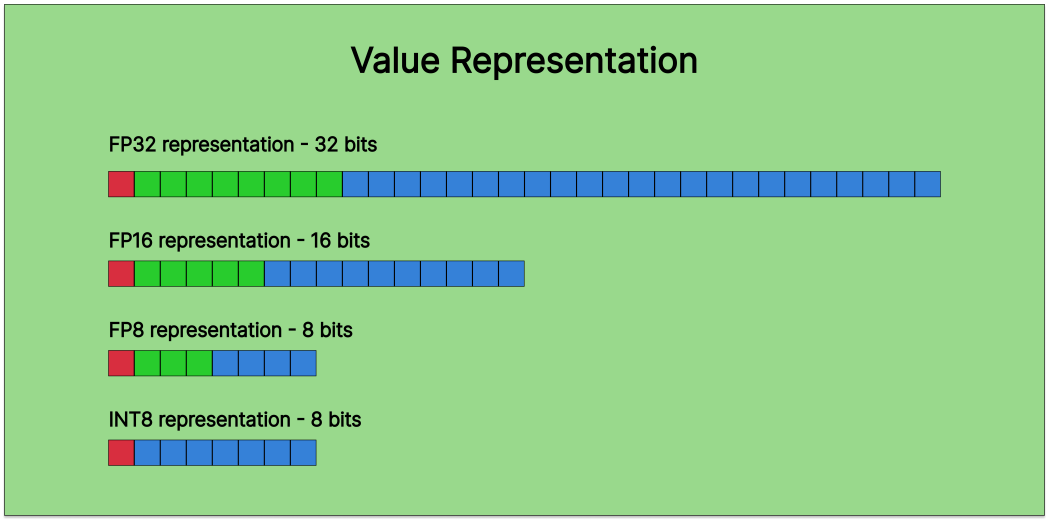

Figure 2. FP32, FP16, FP8, and INT8 representations.

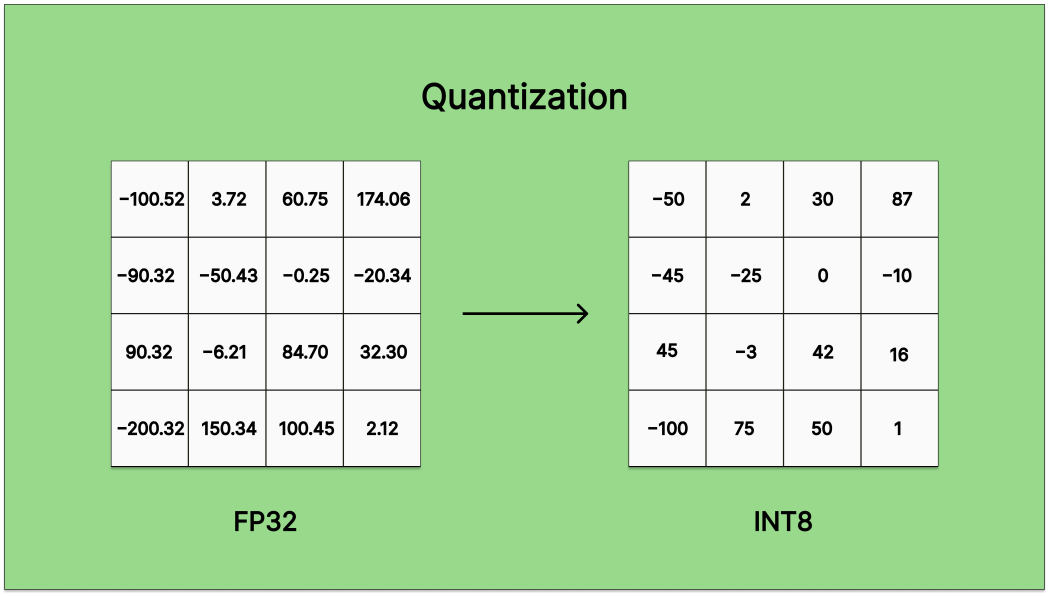

Figure 3. Dynamic range quantization.

The QAT strategy requires more computational effort since it is used while training the model [47].

2.3. Weight Clustering

Weight clustering or weight sharing is an optimization technique that consists of determining a representative value for a cluster or group of weights, thus decreasing the

number of different values to be stored in the network. It means that many different neurons in a network share the same weight [48].

Figure 4 illustrates a basic weight clustering example with sixteen weights being clustered in four groups represented by a color. In this case, four reference values correspond to the mean of the values of each cluster, and each weight is quantized to one of the reference values mentioned above. The green cluster, for example, has five elements with distinct values. The mean of these elements is 0.15, which is the reference value for this cluster. After applying weight sharing, all elements represented in green now have 0.15 as the weight value. For this specific cluster, the distinct parameters are replaced with only one FP32 value and the reference index, which is an integer. Considering all clusters in the example, instead of having sixteen different FP32 values, the network after weight clustering replaces these unique values with four FP32 values, thus optimizing the memory storage.

Figure 4. Simple weight clustering.

The number of clusters chosen affects directly the performance and the memory requirements of a model. An important parameter to be considered is the centroid initialization. The strategy of weight clustering has been assessed for deep learning problems in recent works [49–52].

3. GBRAS-Net

GBRAS-Net is a steganalysis scheme based on convolutional neural networks (CNNs) and was proposed by Reinel et al. [34] in 2021. This deep learning scheme receives an image of 256×256 pixels. The model has a preprocessing layer, which consists of a 2D convolutional layer with 30 filters. This step has 780 non-trainable parameters with preset weights and a custom 3Tanh activation function. The feature extraction stage is composed of various layers: four 2D depthwise convolutional, four 2D separable convolutional, and nine 2D convolutional layers. Furthermore, the model uses batch normalization layers to improve the performance and avoid overfitting, followed by an average pooling layer to reduce the dimensionality. Moreover, the model uses skip connections. At the end of the model's process, global average pooling is carried out, resulting in two values. For the classification stage, these values are input to a softmax activation function, which predicts whether the element is a cover or stego image.

GBRAS-Net was first experimented with in Python 3.8.1 and was developed using the TensorFlow framework version 2.2.0. The authors provided the code on GitHub. The metric

used for the evaluation of the method was the accuracy, and the model was compared with seven other steganalytic methods. At the time, the network had achieved the best accuracy results. The proposed method was implemented and assessed with 10,000 pairs of cover and stego images of BOSSBASE 1.01 for the steganographic algorithm S-UNIWARD, with 73.6% accuracy for 0.2 bpp and 87.1% for 0.4 bpp, and for WOW, with 80.3% accuracy for 0.2 bpp and 89.8% for 0.4 bpp. The data distribution in the experiment was as follows: 4000 pairs of images for training, 1000 pairs for the validation set, and 5000 pairs for testing. In addition, the authors assessed the performance of the proposed model against spatial domain steganography algorithms such as HUGO, HILL, and MiPOD. In terms of model size, GBRAS-Net has one of the smallest storage requirements compared to other steganalysis deep learning models, with only 0.65 MB of occupied memory.

4. Methodology

We applied quantization, pruning, and weight clustering techniques for the optimization of GBRAS-Net and compared the different scenarios in terms of accuracy and occupied memory. We had to update and adjust some parts of the original GBRAS-Net model code to replicate the model and use updated optimization toolkits.

4.1. Computational Requirements

The tests were run on Google Colab with Python 3.10, Tensorflow 2.17.1, and Keras 3.5.0. The machine used was a Ubuntu 22.04.3 LTS, with a Intel(R) Xeon(R) CPU @ 2.20 GHz and RAM of 51 GB (Intel, Santa Clara, CA, USA). Two different GPUs were used, a Tesla T4 with RAM of 15 GB (Tesla Inc., Austin, TX, USA) and an NVIDIA A100-SXM4 with RAM of 40 GB (NVIDIA, Santa Clara, CA, USA).

4.2. Model Compression Framework

To implement quantization, model pruning, and weight clustering, we used the Tensorflow Model Optimization Toolkit (TFMOT) 0.8.0. The models were converted from the keras or hdf5 file format to tflite file formats for quantization.

4.3. Database

We performed our tests on the same database provided by Reinel et al. [34] for comparison purposes. The dataset contains BOSSBASE 1.01 with 10,000 pairs of grayscale images of 256×256.

5. Results

This section presents the results achieved in the simulations. The tables presented in this section summarize all implemented strategies in this work and point out the performance of the model in terms of accuracy and the model's memory consumption for S-UNIWARD 0.4 bpp, S-UNIWARD 0.2 bpp, WOW 0.4 bpp, and WOW 0.2 bpp.

5.1. Dynamic Range Quantization (8 Bits)

This type of quantization means that the weight values will be converted from FP32 to INT8. The activation values and the output and input tensors are stored in FP32. For this implementation, we used the default optimization setting for the TensorFlow lite function.

Tables 1 and 2 show the results of this strategy applied to the S-UNIWARD steganography algorithm. The accuracy loss was 2.36% for S-UNIWARD at 0.4 bpp and 2.72% for S-UNIWARD at 0.2 bpp. The occupied memory achieved was 0.16 MB, corresponding to approximately one quarter of that of the original model.

Table 1. Dynamic range quantization applied to GBRAS-Net for S-UNIWARD at 0.4 bpp.

Strategy	Accuracy	Memory Requirement
GBRAS-Net	87.16%	0.65 MB
Dynamic Range Quantization	84.80%	0.16 MB

Table 2. Dynamic range quantization applied to GBRAS-Net for S-UNIWARD at 0.2 bpp.

Strategy	Accuracy	Memory Requirement
GBRAS-Net	73.60%	0.65 MB
Dynamic Range Quantization	70.88%	0.16 MB

Tables 3 and 4 show the results of the simulations for the WOW algorithm. This strategy applied to the original GBRAS-Net achieved less than a 1% accuracy decrease for the WOW dataset with either 0.4 bpp or 0.2 bpp, reaching 89.05% and 79.32% accuracy, respectively, while exhibiting approximately four times smaller memory requirements than the original model.

Table 3. Dynamic range quantization applied to GBRAS-Net for WOW at 0.4 bpp.

Strategy	Accuracy	Memory Requirement
GBRAS-Net	89.80%	0.65 MB
Dynamic Range Quantization	89.05%	0.16 MB

Table 4. Dynamic range quantization applied to GBRAS-Net for WOW at 0.2 bpp.

Strategy	Accuracy	Memory Requirement
GBRAS-Net	80.30%	0.65 MB
Dynamic Range Quantization	79.32%	0.16 MB

5.2. FP16 Quantization

The FP16 strategy is a quantization technique that applies a conversion from FP32 to FP16. When compared with 8-bit dynamic range quantization, more bits have to be stored after quantization. The weights are then represented in FP16. Tables 5–8 show the FP16 results for S-UNIWARD and WOW for both 0.2 and 0.4 bpp. The GBRAS-Net results are also presented in the aforementioned tables for comparison purposes.

Table 5. FP16 quantization applied to GBRAS-Net for S-UNIWARD at 0.4 bpp.

Strategy	Accuracy	Memory Requirement
GBRAS-Net (Original)	87.16%	0.65 MB
FP16 Quantization	87.29%	0.31 MB

Table 6. FP16 quantization applied to GBRAS-Net for S-UNIWARD at 0.2 bpp.

Strategy	Accuracy	Memory Requirement
GBRAS-Net	73.60%	0.65 MB
FP16 Quantization	74.35%	0.31 MB

Table 7. FP16 quantization applied to GBRAS-Net for WOW at 0.4 bpp.

Strategy	Accuracy	Memory Requirement
GBRAS-Net	89.80%	0.65 MB
FP16 Quantization	89.84%	0.31 MB

Table 8. FP16 quantization applied to GBRAS-Net for WOW at 0.2 bpp.

Strategy	Accuracy	Memory Requirement
GBRAS-Net	80.30%	0.65 MB
FP16 Quantization	80.35%	0.31 MB

Table 5 shows the results of the simulations for S-UNIWARD at 0.4 bpp. The quantized weights model achieved 87.29% accuracy, which represents an increase of 0.13% over the original model. The results for S-UNIWARD at 0.2 bpp, presented in Table 6, show that the accuracy increased by 0.75% when compared with GBRAS-Net, with 74.35%. The results for WOW at 0.4 and 0.2 bpp are shown in Tables 7 and 8, respectively. The accuracy of the model increased slightly, by 0.04% for WOW at 0.4 bpp and 0.05% for WOW at 0.2 bpp, when compared with the original model.

Although the memory requirements are higher than those obtained with dynamic range quantization, they are reduced by approximately half compared to the original GBRAS-Net, with 0.31 MB.

5.3. Weight Clustering

To implement weight clustering, the cluster weight function was used from the TensorFlow optimization toolkit. All layers were clustered, except the first layer of GBRAS-Net. The maximum number of clusters was 32. We assessed the same optimization strategy with four different centroid initializations: density-based [53], K-means++ [54,55], linear, and random. Tables 9–12 show all results for S-UNIWARD and WOW, for both 0.2 and 0.4 bpp.

Table 9. Weight clustering applied to GBRAS-Net for S-UNIWARD at 0.4 bpp.

Strategy	Accuracy	Memory Requirement
GBRAS-Net	87.16%	0.65 MB
Weight Clustering (Density-Based)	87.19%	0.18 MB
Weight Clustering (K-Means++)	86.83%	0.18 MB
Weight Clustering (Linear)	84.28%	0.15 MB
Weight Clustering (Random)	82.03%	0.14 MB

Table 10. Weight clustering applied to GBRAS-Net for S-UNIWARD at 0.2 bpp.

Strategy	Accuracy	Memory Requirement
GBRAS-Net	73.60%	0.65 MB
Weight Clustering (Density-Based)	69.45%	0.18 MB
Weight Clustering (K-Means++)	67.00%	0.18 MB
Weight Clustering (Linear)	64.85%	0.15 MB
Weight Clustering (Random)	68.12%	0.15 MB

Table 11. Weight clustering applied to GBRAS-Net for WOW at 0.4 bpp.

Strategy	Accuracy	Memory Requirement
GBRAS-Net	89.80%	0.65 MB
Weight Clustering (Density-Based)	87.34%	0.18 MB
Weight Clustering (K-Means++)	86.83%	0.18 MB
Weight Clustering (Linear)	85.96%	0.15 MB
Weight Clustering (Random)	89.20%	0.13 MB

Table 12. Weight clustering applied to GBRAS-Net for WOW at 0.2 bpp.

Strategy	Accuracy	Memory Requirement
GBRAS-Net	80.30%	0.65 MB
Weight Clustering (Density-Based)	75.40%	0.18 MB
Weight Clustering (K-Means++)	78.05%	0.17 MB
Weight Clustering (Linear)	76.83%	0.15 MB
Weight Clustering (Random)	75.77%	0.14 MB

Tables 9 and 10 show the results for the S-UNIWARD algorithm at 0.4 bpp and 0.2 bpp, respectively. For S-UNIWARD at 0.4 bpp, when choosing the centroid initialization to be density-based, the model achieved a slight increase of 0.03% in terms of accuracy when compared to GBRAS-Net, while it could save approximately 73% of memory storage. When setting the centroid initialization as K-means++, the model experienced a slight accuracy decrease of 0.33%, with the same storage requirements as for the density-based centroid initialization. Table 10 shows that S-UNIWARD at 0.2 bpp experienced an accuracy decrease of more than 4% for all scenarios.

Tables 11 and 12 show the results for the WOW algorithm at 0.4 bpp and 0.2 bpp, respectively. For the WOW algorithm at 0.4 bpp, when setting the centroid initialization to random, the model obtained accuracy of 89.20%, resulting in a decrease of 0.60% when compared with the original model. This configuration achieved memory storage of 0.13 MB, which was one fifth of that of GBRAS-Net. At 0.2 bpp, the best results in terms of accuracy were obtained when setting the centroid initialization as K-means++, with 78.05%, resulting in a decrease of 2.25% in accuracy and a memory requirement of 0.17 MB.

5.4. Low-Magnitude Pruning

To implement pruning, we used the prune low-magnitude function from the TensorFlow toolkit. We applied pruning for all layers and used 12 epochs for pruning and retraining. We defined the pruning to start from the first epoch with a 10% pruning ratio for all simulations. We assessed the results for eight different pruning ratios across the neurons of the network. The final pruning ratios chosen for the simulations were 20%, 30%, 40%, 50%, 60%, 70%, 80%, and 90%. The results show that the more sparse the network, the lower the occupied memory of the optimized model. The results of this strategy applied to GBRAS-Net for both S-UNIWARD at 0.4 bpp and 0.2 bpp and WOW at 0.4 bpp and 0.2 bpp are shown in Tables 13, 14, 15, and 16, respectively.

Table 13 shows the results for the S-UNIWARD algorithm at 0.4 bpp. None of the simulations resulted in accuracy better than that of the original GBRAS-Net. The best results in terms of accuracy were achieved by defining the pruning ratio as 40% and 50%, with a decrease of 0.18% and 0.04%, respectively. The respective model sizes were greater than the ones seen in the 8-bit dynamic range quantization, FP16 quantization, and weight clustering simulations, with 0.45 MB and 0.39 MB, respectively. At a 90% pruning ratio, accuracy of 82.64% was achieved, with corresponding memory storage of 0.16 MB. The results of the simulations for S-UNIWARD at 0.2 bpp are presented in Table 14. The best results were

obtained by setting the pruning ratio as 50% or 70%, with 73.58% and 74.15% accuracy, respectively. In particular, by setting the pruning ratio to 70%, an increase of 0.55% in accuracy was obtained over the original model, with memory storage of 0.28 MB.

Table 13. Low-magnitude pruning applied to GBRAS-Net for S-UNIWARD at 0.4 bpp.

Strategy	Accuracy	Memory Requirement
GBRAS-Net	87.16%	0.65 MB
Pruning ratio: 20%	86.40%	0.55 MB
Pruning ratio: 30%	86.12%	0.55 MB
Pruning ratio: 40%	86.98%	0.45 MB
Pruning ratio: 50%	87.12%	0.39 MB
Pruning ratio: 60%	85.66%	0.34 MB
Pruning ratio: 70%	85.03%	0.28 MB
Pruning ratio: 80%	84.46%	0.22 MB
Pruning ratio: 90%	82.64%	0.16 MB

Table 14. Low-magnitude pruning applied to GBRAS-Net for S-UNIWARD at 0.2 bpp.

Strategy	Accuracy	Memory Requirement
GBRAS-Net	73.60%	0.65 MB
Pruning ratio: 20%	72.75%	0.54 MB
Pruning ratio: 30%	73.82%	0.50 MB
Pruning ratio: 40%	72.54%	0.45 MB
Pruning ratio: 50%	73.58%	0.39 MB
Pruning ratio: 60%	72.97%	0.34 MB
Pruning ratio: 70%	74.15%	0.28 MB
Pruning ratio: 80%	71.48%	0.22 MB
Pruning ratio: 90%	70.37%	0.16 MB

Table 15. Low-magnitude pruning applied to GBRAS-Net for WOW at 0.4 bpp.

Strategy	Accuracy	Memory Requirement
GBRAS-Net	89.80%	0.65 MB
Pruning ratio: 20%	87.98%	0.54 MB
Pruning ratio: 30%	89.24%	0.50 MB
Pruning ratio: 40%	88.77%	0.45 MB
Pruning ratio: 50%	89.02%	0.39 MB
Pruning ratio: 60%	88.91%	0.34 MB
Pruning ratio: 70%	89.27%	0.28 MB
Pruning ratio: 80%	88.66%	0.22 MB
Pruning ratio: 90%	85.15%	0.16 MB

Table 16. Low-magnitude pruning applied to GBRAS-Net for WOW at 0.2 bpp.

Strategy	Accuracy	Memory Requirement
GBRAS-Net	80.30%	0.65 MB
Pruning ratio: 20%	79.64%	0.54 MB
Pruning ratio: 30%	79.83%	0.50 MB
Pruning ratio: 40%	79.93%	0.45 MB
Pruning ratio: 50%	79.43%	0.39 MB
Pruning ratio: 60%	80.46%	0.34 MB
Pruning ratio: 70%	79.07%	0.28 MB
Pruning ratio: 80%	79.73%	0.22 MB
Pruning ratio: 90%	75.73%	0.16 MB

Table 15 shows the results of the simulations with this strategy applied to WOW at 0.4 bpp. The best result was achieved by setting the pruning ratio at 70%, with accuracy of 89.27%, while obtaining a model size of 0.28 MB. The results for this strategy applied to WOW at 0.2 bpp are presented in Table 16. By defining the pruning ratio as 60%, it was possible to obtain slightly better accuracy compared to GBRAS-Net. In particular, we observed an increase of 0.16% in accuracy when compared with GBRAS-Net, while the model was compressed by approximately two times.

5.5. Low-Magnitude Pruning + Quantization

We combined both pruning and quantization methods. We chose to combine the two better results in terms of accuracy and occupied memory from the previous pruning simulations with both dynamic range quantization and FP16 quantization. Tables 17–20 show the results of this strategy for S-UNIWARD and WOW, for both 0.2 and 0.4 bpp.

Table 17. Low-magnitude pruning combined with quantization applied to GBRAS-Net for S-UNIWARD at 0.4 bpp.

Strategy	Accuracy	Memory Requirement
GBRAS-Net	87.16%	0.65 MB
DRQ + PR: 40%	80.54%	0.14 MB
DRQ + PR: 50%	84.08%	0.13 MB
FP16 + PR: 40%	86.08%	0.24 MB
FP16 + PR: 50%	87.13%	0.21 MB

Table 18. Low-magnitude pruning combined with quantization applied to GBRAS-Net for S-UNIWARD at 0.2 bpp.

Strategy	Accuracy	Memory Requirement
GBRAS-Net	73.60%	0.65 MB
DRQ + PR: 50%	71.85%	0.13 MB
DRQ + PR: 70%	67.87%	0.10 MB
FP16 + PR: 50%	74.19%	0.21 MB
FP16 + PR: 70%	75.19%	0.16 MB

Table 19. Low-magnitude pruning combined with quantization applied to GBRAS-Net for WOW at 0.4 bpp.

Strategy	Accuracy	Memory Requirement
GBRAS-Net	89.80%	0.65 MB
DRQ + PR: 60%	87.08%	0.11 MB
DRQ + PR: 70%	88.25%	0.10 MB
FP16 + PR: 60%	87.44%	0.19 MB
FP16 + PR: 70%	89.49%	0.16 MB

Table 20. Low-magnitude pruning combined with quantization applied to GBRAS-Net for WOW at 0.2 bpp.

Strategy	Accuracy	Memory Requirement
GBRAS-Net	80.30%	0.65 MB
DRQ + PR: 60%	78.73%	0.11 MB
DRQ + PR: 80%	78.12%	0.08 MB
FP16 + PR: 60%	79.55%	0.19 MB
FP16 + PR: 80%	79.12%	0.13 MB

Table 17 presents the results for S-UNIWARD at 0.4 bpp. When combining pruning with a 50% pruning ratio and FP16 quantization, the model achieved accuracy of 87.13%, only 0.03% below that of the original model, while occupying three times less memory. Table 18 shows the results for S-UNIWARD at 0.2 bpp. Two different scenarios obtained better accuracy than the original model. At a 50% pruning ratio combined with FP16 quantization, the model exhibited 0.59% higher accuracy, achieving 74.19%, with a model size of 0.21 MB. A memory requirement that was about four times smaller than that of the original model was obtained, while achieving an increase of 1.59% in accuracy, when implementing a 70% pruning ratio, with the accuracy reaching 75.19%.

Table 19 presents the results when combining quantization and pruning for the WOW algorithm at 0.4 bpp. By setting the pruning ratio at 70%, combined with dynamic range quantization, accuracy of 88.25% was achieved, resulting in a decrease of 1.55% when compared with the original GBRAS-Net model. The corresponding compressed model could save approximately 85% of memory, with 0.10 MB. By combining the same pruning configuration with FP16 quantization, accuracy of 89.49% and a 0.16 MB memory requirement for the model's storage were achieved.

Table 20 presents the results for WOW at 0.2 bpp. By setting the pruning ratio at 80% and combining it with dynamic range quantization, a less memory-demanding model was obtained, with a requirement of 0.08 MB, i.e., an approximately 88% smaller memory storage requirement when compared with the original model. Accuracy of 78.12% was obtained, resulting in a decrease of 2.18% when compared to the original GBRAS-Net.

5.6. Results Analysis

In this subsection, we compare the results obtained for each optimization strategy employed. Figures 5–8 present bar graphs with the results obtained in terms of the accuracy and memory requirements of the optimized model for a selected algorithm at a fixed embedding rate. The accuracy is represented by the blue bars and the memory requirements by orange bars. The bars are arranged in descending order regarding the memory requirements. The left y-axis shows the accuracy value of the model, while the right y-axis shows the memory requirements in MB.

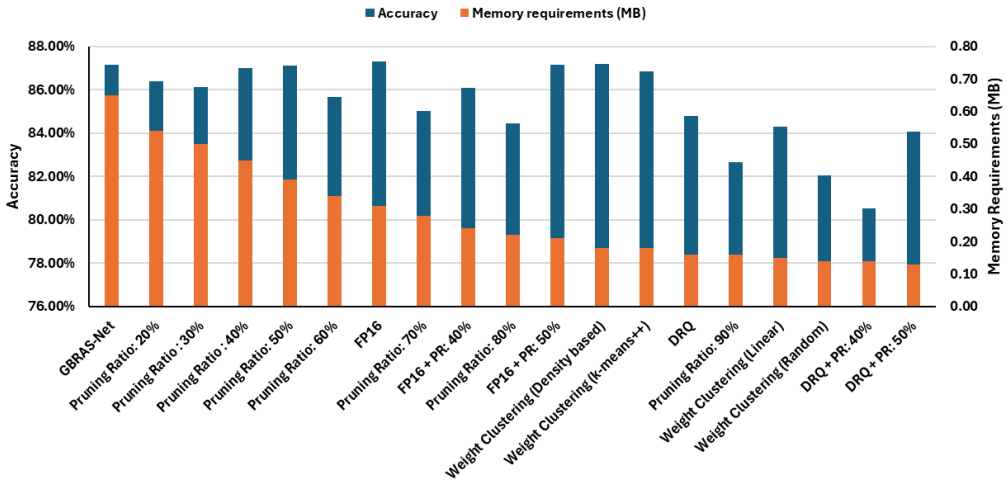

Figure 5. Results in terms of accuracy and memory requirements of all employed optimization techniques for S-UNIWARD at 0.4 bpp.

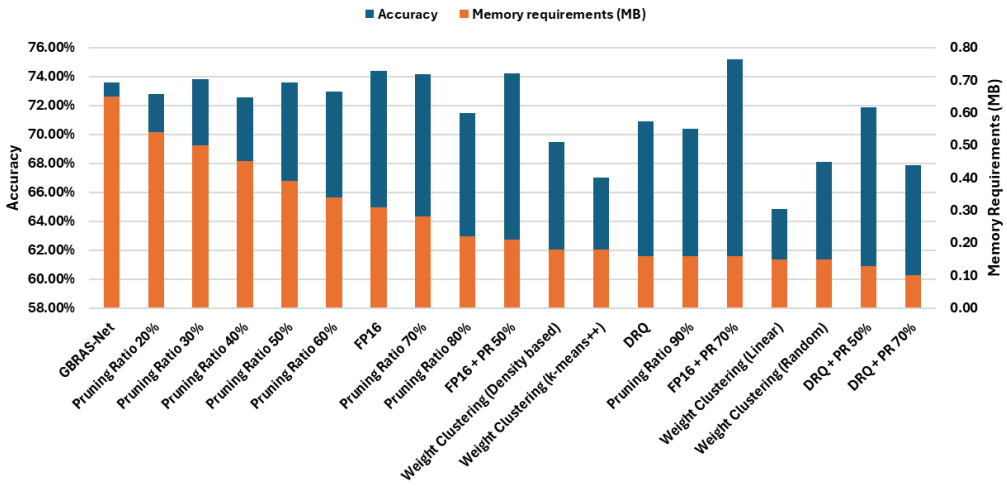

Figure 6. Results in terms of accuracy and memory requirements of all employed optimization techniques for S-UNIWARD at 0.2 bpp.

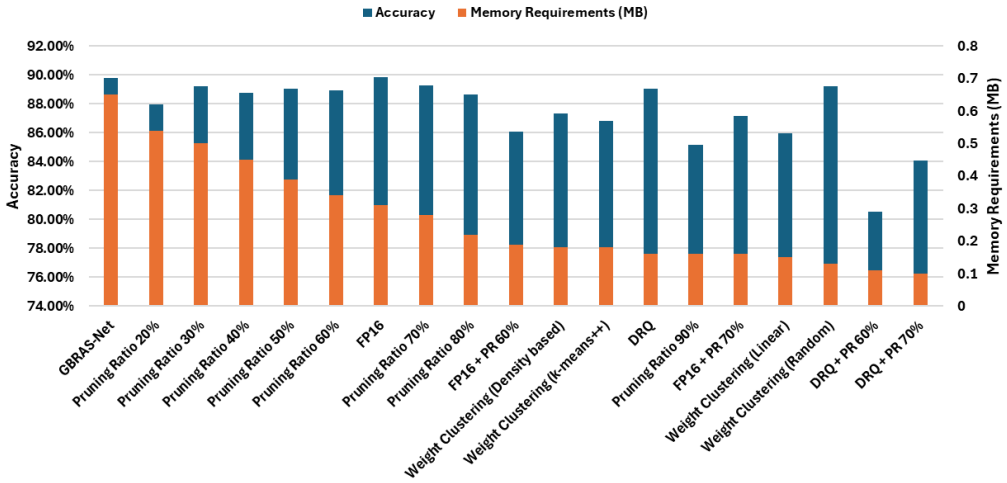

Figure 7. Results in terms of accuracy and memory requirements of all employed optimization techniques for WOW at 0.4 bpp.

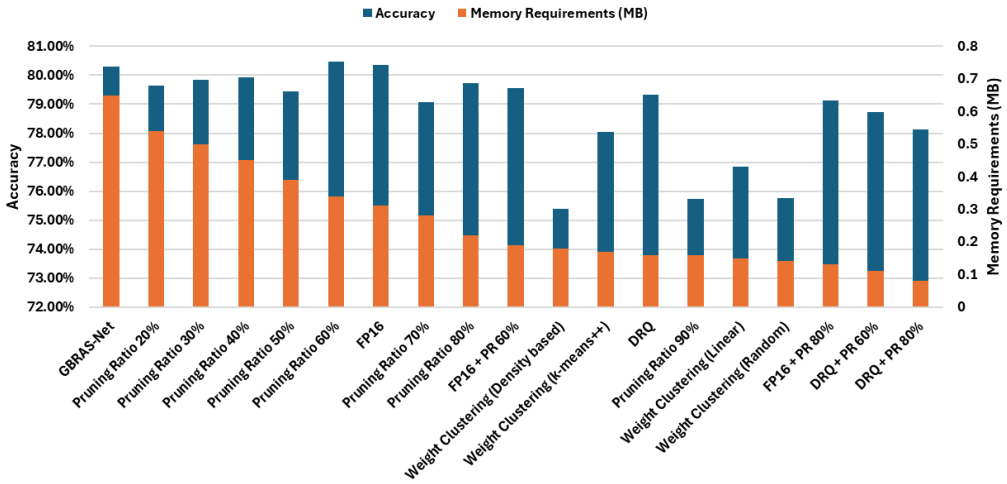

Figure 8. Results in terms of accuracy and memory requirements of all employed optimization techniques for WOW at 0.2 bpp.

Figure 5 shows all results regarding the accuracy and memory requirements of the optimization strategies employed for S-UNIWARD at 0.4 bpp. The best accuracy results were achieved by applying FP16 quantization, FP16 quantization combined with a pruning ratio of 50%, and weight clustering with both density-based and K-means++ centroid initialization. The memory space required by the optimized model when applying FP16 quantization was approximately twice that required when applying each of the other three aforementioned techniques. Considering the relation between accuracy and occupied memory savings, the best results were obtained by applying weight clustering with both K-means++ and density-based centroid initialization and by combining FP16 quantization with a 50% pruning ratio. The weight clustering with density-based centroid initialization proved to be the best choice for this scenario, with a slight increase in accuracy of 0.03% and with approximately 72% less memory required when compared with the original model.

Figure 6 presents the achieved accuracy values and memory requirements of the optimized models when the scenario involved S-UNIWARD at 0.2 bpp. The accuracy of the original model (without the use of optimization strategies) was 73.60%. The results for five different optimized models showed accuracy values that were slightly better when compared with GBRAS-Net, along with savings in the memory required by the model. Accuracy results above 74% were obtained by applying FP16 quantization, a pruning scheme with a 70% ratio, and FP16 combined with a 50% pruning ratio. Considering the relation between the accuracy and the memory requirements of the model, the best results were obtained by applying FP16 quantization combined with pruning of 70%. The accuracy increased by 1.59% when compared to the original model. This strategy resulted in one of the five least space-demanding optimized models when the scenario involved S-UNIWARD at 0.2 bpp, with a size of 0.16 MB, which represents a saving of approximately 75% when compared with the original model.

Figure 7 shows all results regarding the accuracy and memory requirements of the optimization strategies employed for WOW at 0.4 bpp. The accuracy of the original model for this scenario was 89.80%. Six different scenarios presented accuracy values above 89.00%. There was no loss of accuracy when FP16 quantization was applied. The memory saving for this strategy was approximately 52%. The optimized models with dynamic range quantization (0.16 MB) and weight clustering with random centroid initialization (0.13 MB) presented accuracy values of 89.05% and 89.20%, respectively, which represent accuracy losses below 1% when compared with that of GBRAS-Net, while they exhibited smaller memory requirements compared with the optimized model when using FP16 quantization.

Figure 8 presents the achieved accuracy values and memory requirements of the optimized models for WOW at 0.2 bpp. In terms of accuracy, the best values obtained by an optimized model were achieved when applying the pruning ratio of 60%, followed by FP16 quantization. When the scenario involves no loss of accuracy in the model, the aforementioned techniques are recommended. To achieve greater compression, it is recommended to use dynamic range quantization (0.16 MB), which presents an accuracy decrease of 0.98%, or the combination of FP16 quantization with a pruning scheme with a ratio of 80% (0.13 MB), which presents an accuracy decrease of 1.18% when compared with GBRAS-Net.

Figure 9 presents the results obtained in terms of the memory requirements of all optimization strategies employed in this work. The storage requirements of the original model can be reduced from approximately 17% to, at most, 87% when using the considered strategies. The twelve different employed optimization schemes resulted in models with memory requirements that were one quarter of those of the original model, including dynamic range quantization, pruning, weight clustering, and the combination of both dynamic range quantization and FP16 quantization with pruning.

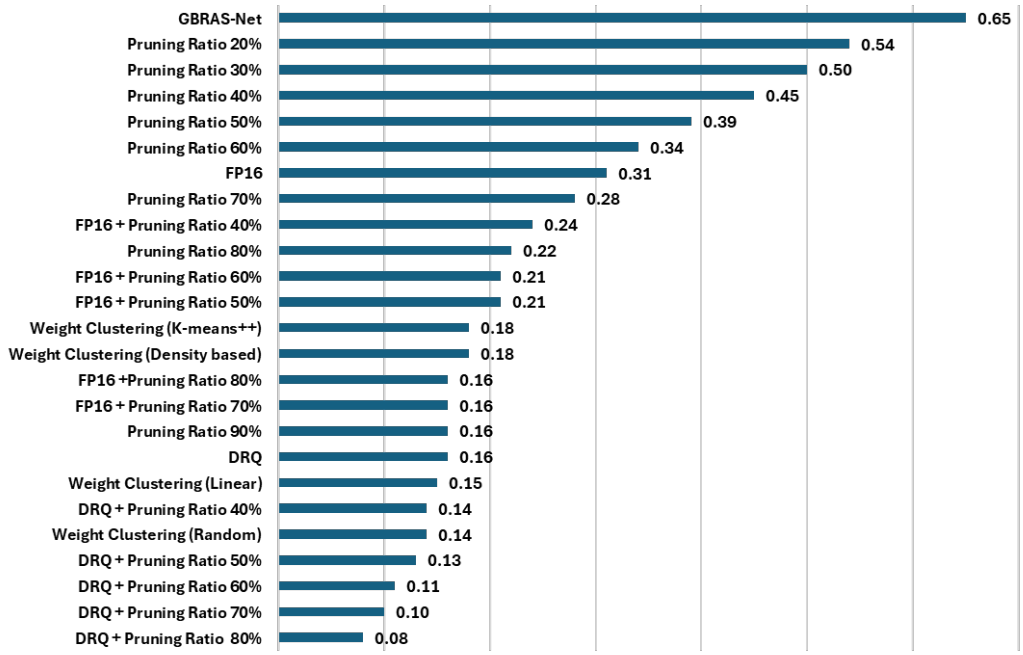

Figure 9. Memory requirements of all assessed optimization strategies.

6. Conclusions

This work addressed a variety of deep learning model optimization techniques, namely model pruning, quantization, and weight clustering, applied to a deep leaning method for image steganalysis (GBRAS-Net). It was observed that these strategies can lead to the same or even better results in terms of accuracy, while reducing the memory requirements for the model's storage by more than three times.

By applying FP16 quantization, we achieved accuracy results that were slightly better than those of GBRAS-Net, while reducing the memory requirements of the model by approximately two times for both the S-UNIWARD and WOW algorithms at either 0.4 bpp or 0.2 bpp.

By applying a weight clustering strategy, we achieved a model that was compressed by four times or more for all scenarios. For S-UNIWARD at 0.4 bpp, we achieved better accuracy results while reducing the memory requirements of the model by approximately four times compared to the original GBRAS-Net. For S-UNIWARD at 0.4 bpp and WOW at both 0.4 bpp and 0.2 bpp, we achieved accuracy results that were slightly worse than those of the original model. For WOW at 0.4 bpp, the decrease in accuracy was only 0.6%. The memory requirements of the compressed model were reduced by four times in this simulation.

We assessed model pruning applied with eight different pruning ratios for both the S-UNIWARD and WOW steganography algorithms at 0.4 bpp and 0.2 bpp. The simulations showed that the more pruned the network, the lower the memory requirements. We achieved slightly better accuracy results when compared with GBRAS-Net for both S-UNIWARD and WOW at 0.2 bpp at a 60% or 70% pruning ratio, respectively.

We combined both dynamic range and FP16 quantization with pruning to achieve even smaller occupied memory values. For S-UNIWARD at 0.2 bpp, accuracy of 75.19% was obtained, increasing by 1.59% when compared with the original GBRAS-Net, while the network was compressed four times. For the WOW algorithm, by applying this combination, we achieved the smallest network sizes. For WOW at 0.2 bpp, we were able to achieve a network with one eighth of the original GBRAS-Net's memory requirement.

We compared the achieved results considering the accuracy impact and the memory savings of the model. For S-UNIWARD at 0.4 bpp, we highlighted the weight clustering strategy with K-means++ centroid initialization, which could maintain the accuracy while leading to a memory saving of 72% in comparison with the original model. For S-UNIWARD at 0.2 bpp, the best result was obtained by applying the combination of FP16 quantization with a pruning scheme at a 70% ratio.

When the scenario involved the WOW algorithm at an embedding rate of 0.4 bpp, FP16 quantization proved to be the best choice in terms of accuracy, while, for greater model compression with a small impact on the accuracy, the best optimized models were those optimized via dynamic range quantization and weight clustering with linear centroid initialization. For an embedding rate of 0.2 bpp, the best results in terms of accuracy and model compression were achieved by applying dynamic range quantization and the combination of FP16 quantization and a pruning scheme at a 60% ratio.

Although the implementation of the optimization techniques was focused on GBRAS-Net, there are other CNNs with greater network sizes. Future work could include addressing optimization strategies for these CNNs. It is important to highlight that there is room for improvement in weight clustering. In this work, four centroid initializations were considered. Other initialization strategies may be addressed or proposed. Future work could include the use of swarm intelligence approaches, e.g., [56], or modified versions of K-means or fuzzy K-means, e.g. [57–60], for the purpose of weight clustering. Another approach is to investigate different values of the maximum number of clusters and their impacts in terms of the accuracy and memory requirements of the model. Another future direction is to investigate the possibility of applying optimization strategies for Vision Transformers applied to steganalysis.

Author Contributions: Conceptualization, G.F., V.S. and F.M.; methodology, G.F., V.S. and F.M.; validation, G.F., V.S. and F.M.; formal analysis, G.F., M.H.d.N.M., V.S. and F.M.; writing—original draft preparation, G.F.; writing—review and editing, G.F., M.H.d.N.M., V.S. and F.M.; supervision, V.S. and F.M.; and project administration, V.S. and F.M. All authors have read and agreed to the published version of the manuscript.

Funding: This work was carried out with the support of the Coordination for the Improvement of Higher Education Personnel—Brazil (CAPES)—Financing Code 001, and the APC was funded by the University of Pernambuco.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Acknowledgments: Thanks to the National Council for Scientific and Technological Development (CNPq) for the support.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Cox, I.; Miller, M.; Bloom, J.; Fridrich, J.; Kalker, T. *Digital Watermarking and Steganography*, 2nd ed.; Morgan Kaufmann Publishers Inc.: San Francisco, CA, USA, 2007.
2. Song, B.; Wei, P.; Wu, S.; Lin, Y.; Zhou, W. A survey on deep-learning-based image steganography. *Expert Syst. Appl.* **2024**, *254*, 124390. https://doi.org/10.1016/j.eswa.2024.124390.
3. Fridrich, J. *Steganography in Digital Media: Principles, Algorithms, and Applications*; Cambridge University Press: Cambridge, UK, 2009.
4. Wu, Z.; Guo, J.; Zhang, C.; Li, C. Steganography and steganalysis in voice over IP: A review. *Sensors* **2021**, *21*, 1032. https://doi.org/10.3390/s21041032.
5. Alneyadi, S.; Sithirasenan, E.; Muthukkumarasamy, V. A survey on data leakage prevention systems. *J. Netw. Comput. Appl.* **2016**, *62*, 137–152. https://doi.org/10.1016/j.jnca.2016.01.008.

6. Desouza, K.C.; Hensgen, T. Semiotic emergent framework to address the reality of cyberterrorism. *Technol. Forecast. Soc. Chang.* **2003**, *70*, 385–396. https://doi.org/10.1016/S0040-1625(03)00003-9.
7. Zielińska, E.; Mazurczyk, W.; Szczypiorski, K. Trends in steganography. *Commun. ACM* **2014**, *57*, 86–95.
8. Wang, H.; Wang, S. Cyber warfare: Steganography vs. steganalysis. *Commun. ACM* **2004**, *47*, 76–82. https://doi.org/10.1145/1022594.1022597.
9. Rallabandi, S.; Sundaram, A.M.; V, K. Generating a multi-OS fully undetectable malware (FUD) and analyzing it afore and after steganography. In Proceedings of the 2022 4th International Conference on Advances in Computing, Communication Control and Networking (ICAC3N), Greater Noida, India, 16–17 December 2022; pp. 1962–1967. https://doi.org/10.1109/ICAC3N56670.2022.10074511.
10. Almehmadi, L.; Basuhail, A.; Alghazzawi, D.; Rabie, O. Framework for malware triggering using steganography. *Appl. Sci.* **2022**, *12*, 8176. https://doi.org/10.3390/app12168176.
11. Vasilellis, E.; Anagnostopoulou, A.; Adamos, K. Hidden realms: Exploring steganography methods in games for covert malware delivery. In *Malware: Handbook of Prevention and Detection*; Springer: Berlin/Heidelberg, Germany, 2024; pp. 355–368.
12. Ibrahim, R.; Kuan, T.S. PRIS: Image processing tool for dealing with criminal cases using steganography technique. In Proceedings of the 2011 Sixth International Conference on Digital Information Management, Melbourne, Australia, 14–16 September 2011; pp. 193–198. https://doi.org/10.1109/ICDIM.2011.6093348.
13. Karampidis, K.; Kavallieratou, E.; Papadourakis, G. A review of image steganalysis techniques for digital forensics. *J. Inf. Secur. Appl.* **2018**, *40*, 217–235. https://doi.org/10.1016/j.jisa.2018.04.005.
14. Bogdanoski, M.; Risteski, A.; Pejoski, S. Steganalysis—A way forward against cyber terrorism. In Proceedings of the 2012 20th Telecommunications Forum (TELFOR), Belgrade, Serbia, 20–22 November 2012; pp. 681–684. https://doi.org/10.1109/TELFOR.2012.6419301.
15. Samuel, H.D.; Kumar, M.S.; Aishwarya, R.; Mathivanan, G. Automation detection of malware and stenographical content using machine learning. In Proceedings of the 2022 6th International Conference on Computing Methodologies and Communication (ICCMC), Erode, India, 29–31 March 2022; pp. 889–894. https://doi.org/10.1109/ICCMC53470.2022.9754063.
16. Alodat, I.; Alodat, M. Detection of image malware steganography using deep transfer learning model. In *Proceedings of the International Conference on Data Science and Applications: ICDSA 2021*; Springer: Singapore, 2022; Volume 2, pp. 323–333.
17. Dalal, M.; Juneja, M. Steganography and steganalysis (in digital forensics): A cybersecurity guide. *Multimed. Tools Appl.* **2021**, *80*, 5723–5771. https://doi.org/10.1007/s11042-020-09929-9.
18. Yari, I.A.; Zargari, S. An overview and computer forensic challenges in image steganography. In Proceedings of the 2017 IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData), Exeter, UK, 21–23 June 2017; pp. 360–364. https://doi.org/10.1109/iThings-GreenCom-CPSCom-SmartData.2017.60.
19. Setiadi, D.R.I.M.; Rustad, S.; Andono, P.N.; Shidik, G.F. Digital image steganography survey and investigation (goal, assessment, method, development, and dataset). *Signal Process.* **2023**, *206*, 108908. https://doi.org/10.1016/j.sigpro.2022.108908.
20. Pevný, T.; Filler, T.; Bas, P. Using high-dimensional image models to perform highly undetectable steganography. In *Information Hiding, Proceedings of the 12th International Conference, IH 2010, Calgary, AB, Canada, 28–30 June 2010*; Böhme, R., Fong, P.W.L., Safavi-Naini, R., Eds.; Springer: Berlin/Heidelberg, Germany, 2010; pp. 161–177.
21. Holub, V.; Fridrich, J. Designing steganographic distortion using directional filters. In Proceedings of the 2012 IEEE International Workshop on Information Forensics and Security (WIFS), Costa Adeje, Spain, 2–5 December 2012; pp. 234–239. https://doi.org/10.1109/WIFS.2012.6412655.
22. Holub, V.; Fridrich, J.; Denemark, T. Universal distortion function for steganography in an arbitrary domain. *EURASIP J. Inf. Secur.* **2014**, *2014*, 1. https://doi.org/10.1186/1687-417X-2014-1.
23. Li, B.; Wang, M.; Huang, J.; Li, X. A new cost function for spatial image steganography. In Proceedings of the 2014 IEEE International Conference on Image Processing (ICIP), Paris, France, 27–30 October 2014; pp. 4206–4210. https://doi.org/10.1109/ICIP.2014.7025854.
24. Muralidharan, T.; Cohen, A.; Cohen, A.; Nissim, N. The infinite race between steganography and steganalysis in images. *Signal Process.* **2022**, *201*, 108711. https://doi.org/10.1016/j.sigpro.2022.108711.
25. Barni, M.; Campisi, P.; Delp, E.; Doërr, G.; Fridrich, J.; Memon, N.; Pérez-González, F.; Rocha, A.; Verdoliva, L.; Wu, M. Information forensics and security: A quarter-century-long journey. *IEEE Signal Process. Mag.* **2023**, *40*, 67–79. https://doi.org/10.1109/MSP.2023.3275319.
26. Ren, Y.; Liu, D.; Liu, C.; Xiong, Q.; Fu, J.; Wang, L. A universal audio steganalysis scheme based on multiscale spectrograms and DeepResNet. *IEEE Trans. Dependable Secur. Comput.* **2023**, *20*, 665–679. https://doi.org/10.1109/TDSC.2022.3141121.
27. Kunhoth, J.; Subramanian, N.; Al-Maadeed, S.; Bouridane, A. Video steganography: Recent advances and challenges. *Multimed. Tools Appl.* **2023**, *82*, 41943–41985. https://doi.org/10.1007/s11042-023-14844-w.

28. Wen, J.; Zhou, X.; Zhong, P.; Xue, Y. Convolutional neural network based text steganalysis. *IEEE Signal Process. Lett.* **2019**, *26*, 460–464. https://doi.org/10.1109/LSP.2019.2895286.
29. Jin, Z.; Feng, G.; Ren, Y.; Zhang, X. Feature extraction optimization of JPEG steganalysis based on residual images. *Signal Process.* **2020**, *170*, 107455. https://doi.org/10.1016/j.sigpro.2020.107455.
30. Agarwal, S.; Kim, C.; Jung, K.H. Steganalysis of context-aware image steganography techniques using convolutional neural network. *Appl. Sci.* **2022**, *12*. https://doi.org/10.3390/app122110793.
31. Agarwal, S.; Jung, K.H. Identification of content-adaptive image steganography using convolutional neural network guided by high-pass kernel. *Appl. Sci.* **2022**, *12*, 11869. https://doi.org/10.3390/app122211869.
32. Płachta, M.; Krzemień, M.; Szczypiorski, K.; Janicki, A. Detection of image steganography using deep learning and ensemble classifiers. *Electronics* **2022**, *11*, 1565. https://doi.org/10.3390/electronics11101565.
33. Croix, N.J.D.L.; Ahmad, T.; Han, F. Comprehensive survey on image steganalysis using deep learning. *Array* **2024**, *22*, 100353. https://doi.org/10.1016/j.array.2024.100353.
34. Reinel, T.S.; Brayan, A.A.H.; Alejandro, B.O.M.; Alejandro, M.R.; Daniel, A.G.; Alejandro, A.G.J.; Buenaventura, B.J.A.; Simon, O.A.; Gustavo, I.; Raúl, R.P. GBRAS-Net: A convolutional neural network architecture for spatial image steganalysis. *IEEE Access* **2021**, *9*, 14340–14350. https://doi.org/10.1109/ACCESS.2021.3052494.
35. Mou, A.; Milanova, M. Performance analysis of deep learning model-compression techniques for audio classification on edge devices. *Sci* **2024**, *6*, 21. https://doi.org/10.3390/sci6020021.
36. Torres-Tello, J.; Ko, S.B. Optimizing a multispectral-images-based DL model, through feature selection, pruning and quantization. In Proceedings of the 2022 IEEE International Symposium on Circuits and Systems (ISCAS), Austin, TX, USA, 27 May–1 June 2022; pp. 1352–1356. https://doi.org/10.1109/ISCAS48785.2022.9937940.
37. Dantas, P.V.; Silva, W.S.; Cordeiro, L.C.; Carvalho, C.B. A comprehensive review of model compression techniques in machine learning. *Appl. Intell.* **2024**, *54*, 11804–11844. https://doi.org/10.1007/s10489-024-05747-w.
38. Han, S.; Pool, J.; Tran, J.; Dally, W.J. Learning both weights and connections for efficient neural networks. In Proceedings of the 28th International Conference on Neural Information Processing Systems, NIPS'15, Montreal, QC, Canada, 7–12 December 2015; Volume 1, pp. 1135–1143.
39. Liang, T.; Glossner, J.; Wang, L.; Shi, S.; Zhang, X. Pruning and quantization for deep neural network acceleration: A survey. *Neurocomputing* **2021**, *461*, 370–403. https://doi.org/10.1016/j.neucom.2021.07.045.
40. Vadera, S.; Ameen, S. Methods for pruning deep neural networks. *IEEE Access* **2022**, *10*, 63280–63300. https://doi.org/10.1109/ACCESS.2022.3182659.
41. Aramdhan, M.R.A.; Mahmudah, H.; Sudibyo, R.W.; Islam, M.M. Optimization of road detection using pruning method for deep neural network. In Proceedings of the 2023 International Electronics Symposium (IES), Denpasar, Indonesia, 8–10 August 2023; pp. 472–478. https://doi.org/10.1109/IES59143.2023.10242577.
42. Wang, Z.; Wen, M.; Xu, Y.; Zhou, Y.; Wang, J.H.; Zhang, L. Communication compression techniques in distributed deep learning: A survey. *J. Syst. Archit.* **2023**, *142*, 102927. https://doi.org/10.1016/j.sysarc.2023.102927.
43. Jiang, H.; Li, Q.; Li, Y. Post training quantization after neural network. In Proceedings of the 2022 14th International Conference on Computer Research and Development (ICCRD), Shenzhen, China, 7–9 January 2022; pp. 1–6. https://doi.org/10.1109/ICCRD54409.2022.9730411.
44. Nooruddin, S.; Islam, M.M.; Karray, F.; Muhammad, G. A multi-resolution fusion approach for human activity recognition from video data in tiny edge devices. *Inf. Fusion* **2023**, *100*, 101953. https://doi.org/10.1016/j.inffus.2023.101953.
45. Rachmanto, R.D.; Sukma, Z.; Nabhaan, A.N.L.; Setyanto, A.; Jiang, T.; Kim, I.K. Characterizing deep learning model compression with post-training quantization on accelerated edge devices. In Proceedings of the 2024 IEEE International Conference on Edge Computing and Communications (EDGE), Shenzhen, China, 7–13 July 2024; pp. 110–120. https://doi.org/10.1109/EDGE62653.2024.00023.
46. *IEEE Std 754-2019 (Revision of IEEE 754-2008)*; IEEE Standard for Floating-Point Arithmetic. IEEE: New York, NY, USA, 2019; pp. 1–84. https://doi.org/10.1109/IEEESTD.2019.8766229.
47. Zhao, X.; Xu, R.; Guo, X. Post-training quantization or quantization-aware training? That is the question. In Proceedings of the 2023 China Semiconductor Technology International Conference (CSTIC), Shanghai, China, 26–27 June 2023; pp. 1–3. https://doi.org/10.1109/CSTIC58779.2023.10219214.
48. Marinó, G.C.; Petrini, A.; Malchiodi, D.; Frasca, M. Deep neural networks compression: A comparative survey and choice recommendations. *Neurocomputing* **2023**, *520*, 152–170. https://doi.org/10.1016/j.neucom.2022.11.072.
49. Jiang, S.; Tao, L. Classification and estimation of typhoon intensity from geostationary meteorological satellite images based on deep learning. *Atmosphere* **2022**, *13*, 1113. https://doi.org/10.3390/atmos13071113.
50. Schiopu, I.; Munteanu, A. Deep learning post-filtering using multi-head attention and multiresolution feature fusion for image and intra-video quality enhancement. *Sensors* **2022**, *22*, 1353. https://doi.org/10.3390/s22041353.

51. Yadav, A.; Fujita, M.; Kumar, B. Resource-efficient DL model inference with weight clustering and zero-skipping. In Proceedings of the 2024 21st International SoC Design Conference (ISOCC), Sapporo, Japan, 19–22 August 2024; pp. 358–359. https://doi.org/10.1109/ISOCC62682.2024.10762251.
52. Cho, M.; Vahid, K.A.; Fu, Q.; Adya, S.; Mundo, C.C.D.; Rastegari, M.; Naik, D.; Zatloukal, P. eDKM: An efficient and accurate train-time weight clustering for large language models. *IEEE Comput. Archit. Lett.* **2024**, *23*, 37–40. https://doi.org/10.1109/LCA.2024.3363492.
53. Castellano, G.; Cotardo, E.; Mencar, C.; Vessio, G. Density-based clustering with fully-convolutional networks for crowd flow detection from drones. *Neurocomputing* **2023**, *526*, 169–179. https://doi.org/10.1016/j.neucom.2023.01.059.
54. Arthur, D.; Vassilvitskii, S. K-means++: The advantages of careful seeding. In Proceedings of the SODA '07: Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms, New Orleans, LA, USA, 7–9 January 2007; pp. 1027–1035. Available online: https://dl.acm.org/doi/10.5555/1283383.1283494 (accessed on 15 December 2024).
55. Ikotun, A.M.; Ezugwu, A.E.; Abualigah, L.; Abuhaija, B.; Heming, J. K-means clustering algorithms: A comprehensive review, variants analysis, and advances in the era of big data. *Inf. Sci.* **2023**, *622*, 178–210. https://doi.org/10.1016/j.ins.2022.11.139.
56. Severo, V.; Ferreira, F.B.S.; Spencer, R.; Nascimento, A.; Madeiro, F. On the initialization of swarm intelligence algorithms for vector quantization codebook design. *Sensors* **2024**, *24*, 2606. https://doi.org/10.3390/s24082606.
57. Mata, E.; Bandeira, S.; De Mattos Neto, P.; Lopes, W.; Madeiro, F. Accelerating families of fuzzy k-means algorithms for vector quantization codebook design. *Sensors* **2016**, *16*, 1963. https://doi.org/10.3390/s16111963.
58. de Maeyer, R.; Sieranoja, S.; Fränti, P. Balanced k-means revisited. *Appl. Comput. Intell.* **2023**, *3*, 145–179. https://doi.org/10.3934/aci.2023008.
59. Malinen, M.I.; Fränti, P. Balanced k-means for clustering. In Proceedings of the Structural, Syntactic, and Statistical Pattern Recognition, Joensuu, Finland, 20–22 August 2014; pp. 32–41. https://doi.org/10.1007/978-3-662-44415-3_4.
60. Malinen, M.I.; Fränti, P. Fixed-sized clusters k-means. *arXiv* **2025**, arXiv:2501.16113. https://doi.org/10.48550/arXiv.2501.16113.